IMPLEMENTING POST-QUANTUM SECURE EXOTIC SIGNATURE SCHEMES IN BLOCKCHAINS

A dissertation submitted to The University of Manchester
for the degree of Master of Science
in the Faculty of Science and Engineering

2026

Royce Steven

Department of Computer Science

Contents

List of Tables

List of Figures

Abstract

Implementing Post-Quantum Secure
Exotic Signature Schemes in Blockchains
Royce Steven
A dissertation submitted to The University of Manchester
for the degree of Master of Science, 2026

Blockchains authenticate transactions with elliptic-curve signatures, which a large quantum computer would break. Basic post-quantum signatures are now standardised, but the *exotic* signatures that give blockchains their advanced features — and in particular *adaptor signatures*, the cryptographic core of atomic swaps and payment channels — remain largely paper-only in the post-quantum setting. This dissertation implements and evaluates one such exotic scheme, LAS, a lattice-based adaptor signature, by reusing the CRYSTALS-Dilithium reference primitives, and demonstrates it end-to-end in a post-quantum atomic swap.

The artefact is built *additively* on Dilithium. The entire lattice core — polynomial arithmetic, the number-theoretic transform, and sampling based on SHAKE (an extendable-output hash function) — is reused unchanged, verified by a file-level diff against a pristine baseline; not a single upstream function was modified. The adaptor layer (PreSign, PreVerify, Adapt, Extract), a bit-packed wire encoding with a validating decoder, and the atomic-swap demonstration are added as new code. The implementation passes functional tests over 1000 iterations on three parameter sets, exact serialization round-trips, a tamper test in which all 4640 single-byte signature flips are rejected, and pinned known-answer tests. An independent second implementation in Rust, built the same way on the Rust ML-DSA crate, reproduces the wire format and the pinned known-answer digest

byte-for-byte and independently confirms the benchmark conclusions.

The primary, fair comparison is LAS against its own simplified Dilithium-style base, using the same parameters and primitives, which isolates the cost of the adaptor layer; the optimised CRYSTALS-Dilithium reference is reported only as context, and a classical Elliptic Curve Digital Signature Algorithm (ECDSA) adaptor as a functionality-matched "price of post-quantum" baseline. Every timing is a mean over repeated independent runs and is reported with its standard deviation. The data show that the adaptor layer is almost free in computation — PreSign, PreVerify, and Adapt each add only single-digit overhead (at most +10.5%) over the basic operation they mirror, at every parameter set, and Extract is the cheapest operation. The real cost is *communication*: the signature is 99.3–99.7% lattice response vector z, and an adaptor swap additionally transmits a public-key-sized statement. The atomic-swap demonstration settles both legs of a two-party, two-chain exchange and asserts that a pre-signature is unspendable until the witness is revealed.

The dissertation concludes that a working, tested, end-to-end post-quantum exotic signature is achievable by reuse of standardised primitives, and identifies tighter packing and migration to the larger modulus of the original LAS construction as the principal avenues for future work.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Chapter 1

Introduction

This chapter sets out why post-quantum exotic signatures matter for blockchains, what this project builds, and how the dissertation is organised. It first establishes the context, then reviews the necessary literature, then states the aim and objectives.

1.1 Context and motivation

Public blockchains authorise every transaction with a digital signature, almost always the Elliptic Curve Digital Signature Algorithm (ECDSA), Schnorr, or Boneh–Lynn–Shacham (BLS) signatures over an elliptic curve. The security of all three rests on the hardness of the elliptic-curve discrete logarithm problem. Shor's algorithm solves that problem in polynomial time on a sufficiently large quantum computer [17], so a future quantum adversary could forge signatures and spend other users' funds. "Post-quantum" (PQ) cryptography replaces these assumptions with problems believed hard even for quantum computers, such as those over lattices.

The U.S. National Institute of Standards and Technology (NIST) has standardised *basic* PQ signatures, notably ML-DSA (the Module-Lattice-Based Digital Signature Algorithm), derived from CRYSTALS-Dilithium [13, 5]. A basic signature authenticates a message and nothing more. Blockchains, however, depend on *exotic* signatures that add functionality on top of a basic scheme: multi-signatures, aggregate signatures, threshold signatures, and *adaptor signatures*. These exotic schemes are what give blockchains scalable consensus, compact multi-party authorisation, and advanced payment protocols. A recent survey documents that, while

basic PQ signatures are maturing, their exotic counterparts largely lack efficient, practical implementations in a blockchain setting [3]. Closing that gap — turning paper designs into tested, measured, blockchain-facing artefacts — is the problem this project addresses.

This dissertation focuses on the *adaptor signature* as its exotic scheme. An adaptor signature ties the act of publishing a valid signature to the simultaneous revelation of a secret witness [14, 1]. This single primitive underpins "scriptless" atomic swaps and payment-channel networks: it lets two parties exchange assets, even across different chains, so that either both transfers happen or neither does, without trusted intermediaries or heavy on-chain scripts. In the classical setting these constructions are mature and deployed; in the post-quantum setting they are mostly paper-only, which makes the adaptor signature a representative and high-value exotic scheme to implement and evaluate.

The adaptor signature is best understood through the history of the atomic swap. A cross-chain exchange first relied on custodial intermediaries; hash-time-locked contracts (HTLCs) removed the custodian by locking both transfers under the preimage of one hash, so that claiming one leg reveals the preimage that unlocks the other. HTLCs, however, need explicit on-chain script support, place the common hash on both chains — publicly linking the two legs and, in multi-hop payments, every hop of a path — and enlarge the transactions. The adaptor signature answers these limitations [14, 1]: it moves the lock *into the signature itself*, so settlement transactions look like ordinary single-signer payments — no script beyond signature verification, no on-chain linkage between the legs, no extra on-chain bytes — while retaining the atomicity guarantee, and revealing the witness only to the counterparty who holds the matching pre-signature.

1.2 Subject area and related work

This section reviews only the work needed to understand the problem and justify the approach; it is deliberately not an exhaustive survey, in line with the report guidance to favour depth over breadth.

Exotic signatures for post-quantum blockchains. The motivating survey [3] catalogues exotic signature families — multi-, aggregate, threshold, ring, blind, and adaptor signatures — and identifies the shortage of efficient PQ-secure

blockchain implementations as an open challenge. This project contributes one such implementation and evaluation.

Lattice signatures and Dilithium. CRYSTALS-Dilithium is a lattice signature in the "Fiat–Shamir with aborts" family [5, 11]. Its security reduces to the Module Learning With Errors (Module-LWE) and Module Short Integer Solution (Module-SIS) problems. The scheme samples a masked commitment, derives a challenge by hashing, computes a response, and *rejects and retries* when the response would leak the secret — the rejection-sampling step that characterises the family. Its reference implementation provides the polynomial arithmetic, number-theoretic transform (NTT), and SHAKE-based sampling (SHAKE is an extendable-output hash function) that this project reuses.

Adaptor signatures. Adaptor signatures were introduced informally as "scriptless scripts" [14] and later formalised, with applications to channels and swaps [1]. The abstraction adds four algorithms to a base signature — PreSign, PreVerify, Adapt, Extract — relative to a hard relation (Y, y): a pre-signature can be *adapted* into a full signature by anyone who knows the witness y, and publishing the full signature lets anyone holding the pre-signature *extract* y. Anonymous multi-hop locks generalise this to payment paths [12].

Post-quantum adaptor signatures. Esgin, Ersoy and Erkin proposed LAS, a lattice-based adaptor signature built on a Dilithium-style base, together with PQ payment-channel constructions [6]. LAS is the design this project implements. Their work is primarily theoretical (definitions, a construction, and security proofs); a practical, benchmarked, blockchain-facing implementation is what this dissertation contributes. Recent work such as poqeth studies how PQ primitives can be integrated and costed on Ethereum [9], providing a template for the on-chain side.

Classical baseline. The natural answer to "what does post-quantum cost" is a comparison with a *classical adaptor* signature — not merely plain ECDSA. Mature ECDSA-adaptor implementations exist [2]; this project benchmarks against one, and against Dilithium itself, taking care (see chapter 2) to state security parameters explicitly so that the comparisons are fair.

1.3 Project category, aim, and objectives

Category. This is a *systems implementation and evaluation* project grounded in existing research, not a proposal of a new cryptographic protocol. Its emphasis, following the project brief, is practical implementation and benchmarking rather than theoretical design. The contribution is a reliable, tested, and benchmarked realisation of an existing exotic scheme (LAS, [6]) and its demonstration in a blockchain atomic-swap application — comparable in spirit to releasing a working signature library rather than inventing a new scheme. A genuinely new research question would require adding new functionality on top of adaptor signatures (for example, functional adaptor signatures); that is noted as future work but is out of scope here.

Aim. To implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a post-quantum blockchain atomic-swap application.

Objectives.

1. Review post-quantum and exotic-signature literature sufficiently to justify selecting the adaptor signature LAS and the Dilithium primitives.

2. Implement LAS additively on the Dilithium reference, reusing the lattice core unchanged and adding the adaptor operations and a byte-level wire encoding.

3. Validate correctness and robustness through functional, serialization, tamper, and known-answer tests, including cross-validation by a second, independent implementation checked against the same known-answer digest.

4. Benchmark LAS fairly — at explicitly stated parameters, with repeated runs and reported variance — against a simplified Dilithium baseline at identical parameters, the NIST-optimised Dilithium, and a classical ECDSA-adaptor baseline, separating computation from communication cost and reporting key-generation time, public-key size, signature size, signing time, and verification time.

5. Demonstrate an end-to-end post-quantum atomic swap using the implementation — following the protocol of [6] verbatim, including its zero-knowledge proof of knowledge π — and discuss its feasibility.

1.4 Dissertation structure

Chapter 2 describes the methods: the LAS construction, the reuse-based implementation strategy, the testing approach, and the benchmarking methodology. Chapter 3 reports the test and benchmark results and discusses their validity. Chapter 4 evaluates the work against the objectives and reflects on the approach. Chapter 5 concludes and proposes future work.

Chapter 2

Methodology

This chapter explains how the project work was done: the construction implemented, the reuse-based implementation strategy, the testing regime, the independent Rust port that cross-validates the implementation, and — importantly for a fair evaluation — the benchmarking methodology and the parameter choices that make the comparisons defensible. The atomic-swap demonstration and its simulated-ledger model are methodologically light and are described together with their results in section 3.5.

2.1 The LAS construction

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [6, 5]. All objects live in the ring $R_q = \mathbb{Z}_q[X]/(X^d + 1)$. The public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n \times (n+\ell)}$. A key pair is a short secret r and its image $t = A \cdot r$; the hard relation (Y, y) reuses exactly the same structure, so a statement is "just another public key" and recovering its witness is a Module-SIS/LWE problem. Crucially, *key generation is shared*: LAS reuses the basic signature's KeyGen unchanged, and the statement/witness pair (Y, y) is produced by that very same generator. LAS is therefore exactly the basic signature — KeyGen, Sign, Verify — plus the four adaptor operations below, which is why every comparison in this report pairs a LAS operation with the basic operation it extends.

The base signature samples a masked commitment w, forms a challenge $c = H(pk, w, M)$, computes a response, and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$ (eq. (2.1)). The adaptor extension [6] adds four operations relative to a statement Y:

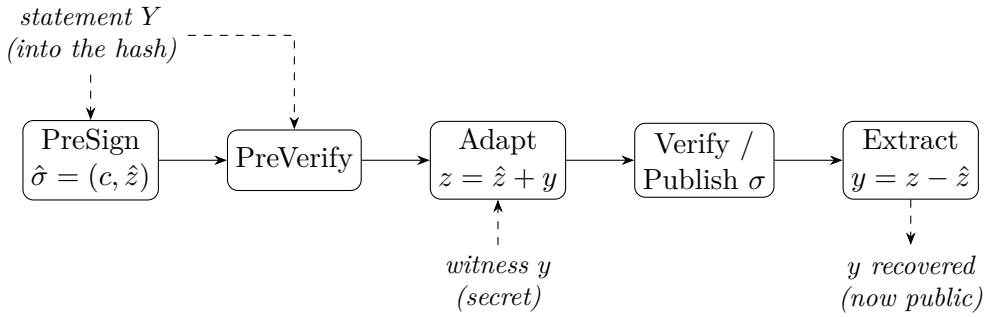

Figure 2.1: The LAS adaptor data flow. The statement Y is folded into the challenge hash during PreSign and PreVerify; the witness y is added by Adapt to turn the pre-signature into a basic signature; and publishing that signature lets anyone holding the pre-signature run Extract to recover y. This last step is what makes an atomic swap atomic.

- PreSign folds the statement into the hash, $c = H(pk, w + Y, M)$, and rejects at the *tighter* bound $\gamma - \kappa - 1$;
- PreVerify recomputes $w' = A\mathbf{z} - c\,t$ and checks $c = H(pk, w' + Y, M)$;
- Adapt adds the witness, $\mathbf{z} \leftarrow \mathbf{z} + y$, producing a signature that the basic verifier accepts;
- Extract recovers $y = \mathbf{z} - \hat{\mathbf{z}}$ from a signature and its pre-signature.

$$\text{accept iff } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\, d\,(n + \ell). \tag{2.1}$$

The single subtle design point is the bound budget. PreSign must reject at the tighter bound $\gamma - \kappa - 1$. The witness is ternary, so $\|y\|_\infty \le 1$, and after Adapt adds it the response satisfies $\|\mathbf{z}\|_\infty \le \gamma - \kappa$ and clears eq. (2.1). Loosening PreSign to $\gamma - \kappa$ would let adapted signatures exceed the bound, and basic verify would then reject every adapted signature. Figure 2.1 shows the four-operation data flow and, crucially, where the statement Y enters and where the witness y is revealed.

Table 2.1: Source files of the implementation, classified by how the reference is used. The lattice core is reused unchanged; the only modified file is the `Makefile`; the scheme, serialization, application, and evaluation are new.

Category	Files	Role
Reused unchanged	`poly/polyvec/ntt/reduce/rounding`, `packing`, `sign`, `fips202`, `params`	the entire lattice core
Modified	`Makefile`	compile new targets
Added	`las.{c,h}`, `serialize.{c,h}`, `relation_zk.{c,h}` (+ LaZer bridge), `amhl`, `chain`, tests, benchmarks	this work

2.2 Implementation strategy: reuse, do not reinvent

The guiding principle is that an exotic scheme equals a basic scheme plus extra functions, so the lattice arithmetic should be *reused*, not rewritten. The implementation is therefore built additively on the upstream CRYSTALS-Dilithium reference implementation [5, 4], used at a pinned commit; the same strategy is later applied a second time, in Rust, on the Rust ML-DSA crate (section 2.5). Neither implementation is itself "the reference": both are *modified, simplified* ML-DSA-style bases carrying the LAS layer of [6], and throughout this report they are named the *C implementation* and the *Rust implementation* (or Rust port).

Table 2.1 summarises the classification: no upstream source function is modified, so the security-critical lattice arithmetic is exactly the audited reference code, and every line specific to the adaptor scheme is new and isolated. This both reduces the surface for implementation error and makes the boundary between reused and new code unambiguous. The same posture extends to the two vendored third-party libraries: the classical baseline reuses `secp256k1-zkp` [2] unmodified, and the atomic swap's proof of knowledge π (section 3.5) reuses the LaZer lattice zero-knowledge library [10] unmodified, behind a single-file bridge.

Data types and the matrix product. LAS is a self-contained module (`las.{c,h}`) over the reference's degree-d polynomial type. A key pair is a ternary secret r of $n + \ell$ polynomials and its image $t = A\,r$ of n polynomials; a

(pre-)signature is a challenge c and a response $\mathbf{z}$ of $n + \ell$ polynomials; a statement/witness (Y, y) has exactly the key-pair shape. The public matrix has the form $A = [\, I_n \mid A' \,]$, so the product $A\,v = v_{\text{top}} + A' v_{\text{bot}}$ adds the identity block directly and multiplies only A' in the NTT domain. This both saves work and matches exactly how the reference computes As_1 in key generation.

Hash, challenge, and samplers. The random oracle H is built from SHAKE256: it absorbs a canonical encoding of the public key, then the commitment (w for Sign, $w + Y$ for PreSign), then the message, and squeezes a seed. The challenge sampler is the reference's `SampleInBall` with weight κ (table 2.2; e.g. 60 at the LAS-2020/845 reference set, 49 at the target Simplified Dilithium-III), which guarantees $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary sampler for secrets and witnesses (two bits per coefficient), and a uniform sampler over $[-\gamma, \gamma]$ for the mask (an 18-bit rejection sampler). The infinity-norm rejection checks reuse the reference's `poly_chknorm` directly.

Simplifications relative to the full Dilithium scheme. Following the LAS construction [6], the base signature omits the optimisations of the full Dilithium scheme — the hint vector, Power2Round, and high/low-bit decomposition — and hashes the full commitment w rather than its high bits. This keeps the algebra transparent at the cost of larger keys and signatures (quantified in chapter 3). The module depends on no mode-specific constant, so the same code runs at any of the parameter sets used in the evaluation.

Modulus (a deliberate deviation). The LAS construction [6] specifies $q \approx 2^{24}$. Reusing the reference NTT fixes the root-of-unity table to $q = 8\,380\,417 \approx 2^{23}$, so this implementation uses that modulus. Since $q > 2\gamma$, every intermediate value is represented faithfully and correctness is unaffected; only the concrete Module-SIS/LWE security margin differs from the larger-modulus parameter set. Migrating to $q \approx 2^{24}$ would require a new NTT table and is treated as future work (section 5.2).

2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs. A bit-packed wire encoding and a *validating* decoder were implemented, together with a verify-from-bytes entry point (`las_verify_packed`) that an on-chain verifier would consume. The packed field widths (used for the size analysis in chapter 3) are: public key / statement at 23 bits/coefficient, ternary secret / witness at 2 bits/coefficient, and the response z at 18 bits/coefficient.

A verifier cannot trust its input, so the decoder is *defensive*: it rejects a public-key coefficient $\geq q$, the invalid ternary code, and any response field outside the 18-bit band. The encoder is symmetric, rejecting a non-ternary secret or a response exceeding $\gamma - \kappa$. This is the realistic interface a Solidity, precompile, or zero-knowledge-circuit verifier would expose, and it is the object the tamper test in section 3.1 attacks.

Challenge encoding: seed, not polynomial. The wire format does not carry the challenge polynomial c itself. Following the encoding of FIPS 204 [13], a signature carries only the 32-byte commitment hash $\tilde{c}$ — the SHAKE256 output from which c is derived — and every consumer re-derives $c = \mathsf{SampleInBall}(\tilde{c})$ after decoding. This is a computation-versus-storage trade-off: storing the expanded polynomial would save the re-derivation at every decode, while storing the seed keeps the signature smaller and matches the standard's format. In deployments where the same signature is decoded repeatedly, the expanded challenge can be cached after the first derivation, so the re-derivation is a one-time cost per object rather than a per-use one.

2.4 Testing methodology

Correctness and robustness were established with four test types.

1. **Functional tests** run 1000 iterations per Dilithium mode (2, 3, and 5), each with fresh public parameters, keys, and message. Every iteration runs the full adaptor cycle and asserts: the pre-signature pre-verifies; the pre-signature *fails* basic verify (the statement-binding tripwire); the adapted signature verifies; and the extracted witness equals the original *exactly*. A

basic sign/verify round-trip and a one-bit-forgery rejection are also checked. Passing all 1000 iterations at every mode is the bar for functional correctness.

2. **Serialization tests** exercise the byte encoding over 256 random instances, asserting exact round-trip for keys and all signature variants.

3. **A tamper test** flips every one of the 4640 bytes of a valid packed signature, one at a time, and asserts that all 4640 flips break verification; it also checks that the validating decoder rejects malformed bytes.

4. **Known-answer tests** fix all inputs, run the deterministic application programming interface (API), fold the packed bytes of every object into a single SHAKE256 digest, and compare it against a pinned 32-byte value. Any unintended change to key generation, signing, the adaptor algebra, or the serialization flips the digest. The digest is stable across machines and compilers, so an independent on-chain verifier can be cross-checked against it.

All tests pass, and the implementation compiles with no warnings under a strict set of compiler diagnostics.

2.5 An independent second implementation: the Rust port

The complete scheme — the ordinary signature, the four adaptor operations, the wire encoding, and the deterministic known-answer interface — was implemented a second time, in Rust. The port follows the same reuse principle as the C implementation: it is layered on a fixed version of the `fips204` crate, a Rust implementation of ML-DSA [16], reusing that crate's polynomial arithmetic, number-theoretic transform, and SHAKE-based sampling, and adding the LAS layer as new, self-contained modules. The wire format is byte-identical to the C encoder's by construction. Figure 2.2 shows the resulting structure of the repository at a glance: two parallel implementations, each a new LAS layer over an unmodified base of reused primitives, tied together by the shared known-answer digest.

The port serves three methodological purposes. First, *cross-validation*: the strongest practical argument that a cryptographic implementation is correct, short

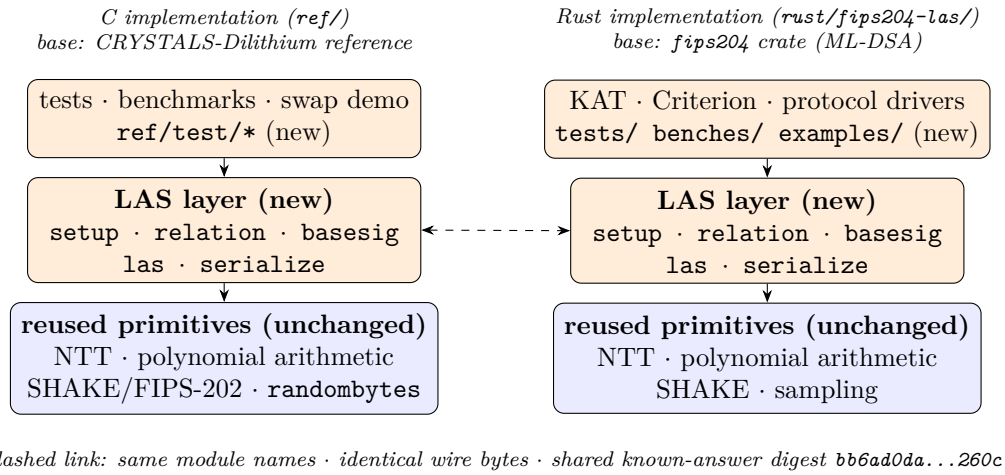

Figure 2.2: Repository structure. Both implementations have the same shape: a new, self-contained LAS layer (orange) — the same five modules under the same names — built on an *unmodified* base of reused lattice primitives (blue), with the tests and benchmarks on top. The two columns are tied together by construction: identical wire bytes and the shared pinned known-answer digest (`bb6ad0da...260c`). This is the high-level map for reading the code; the per-file classification is table 2.1.

of a formal proof, is a second implementation written against the specification of the LAS construction [6] rather than against the first implementation's quirks, agreeing on pinned test vectors. Both implementations reproduce the same known-answer digest — a single SHAKE256 hash folded over the packed bytes of every object of four deterministic vectors — so their samplers, algebra, and encoders agree exactly (section 3.3). Second, *an independent benchmark methodology*: the Rust ecosystem's standard statistical harness, Criterion.rs [8] (warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals), measures the same operations without sharing a line of timing code with the hand-rolled C harness, so agreement between the two defends the C numbers against harness artefacts. Third, *deployment realism*: a memory-safe language is the plausible vehicle for any production use of the scheme, and the port shows the design does not depend on C-specific constructions.

To make the C and Rust timings directly comparable, the two protocol drivers mirror each other exactly: the same repetition scheme, the same fixed public-parameter seed, the same fixed message, the same success-path gating (the correctness contract of section 2.4 is asserted before any timing starts), and the same directly counted rejection-loop statistics. The Criterion harness then

provides the independent statistical cross-check on top. One caveat qualifies the port's independence: where the LAS construction [6] leaves an engineering choice open, the Rust implementation deliberately adopts the C implementation's choice rather than making its own. The concrete instance is the rejection check inside the signing loop, which *aborts early* at the first out-of-bound coefficient instead of always scanning the full response vector; both implementations share this early-abort behaviour so that the two languages time the same algorithm, and any such tie is verified rather than assumed by the shared known-answer digest and the attempt-count gate below.

2.6 Benchmarking methodology and fair comparison

The evaluation separates two distinct costs and never conflates them: *computation* cost (wall-clock time per operation) and *communication* cost (bytes transmitted or stored). Each is measured as follows.

Measurement boundaries: two timing tiers. Reported timings depend critically on where the measurement boundary is drawn — undocumented boundaries are a recognised benchmarking pitfall for lattice signatures [15]. Every timing in this report therefore belongs to one of two explicitly defined tiers, and comparisons are only ever made within a tier:

- the **core tier**: in-memory structures in, structures out. This boundary isolates the cryptographic arithmetic (sampling, NTT products, hashing, rejection) and is the right tier for asking what the adaptor *algebra* costs;
- the **packed tier** (named after the implementation's `*_packed` byte interfaces): packed wire bytes in, packed wire bytes out, including the validating decode of every input and the encode of every output (section 2.3). This is the serialization-inclusive boundary — the per-call cost a wire or on-chain consumer pays when nothing is cached. It is deliberately *not* called an end-to-end or protocol cost: it covers one operation's byte boundary, not the multi-message swap protocol.

Both tiers are measured for the base signature and for LAS, in C and in Rust, and wherever a timing comparison between the base scheme and LAS is presented,

both tiers are shown. The classical baseline cannot be measured at either tier cleanly, which forces a third, explicitly-named boundary:

- the **hybrid native-API boundary** (classical baseline only): timing at the boundary the `secp256k1-zkp` library itself exposes, reused unmodified. This boundary is *hybrid* because it mixes the two tiers *per operation*: the four adaptor operations — PreSign, PreVerify, Adapt, and Extract — serialize or parse the 162-byte pre-signature *inside* the timed call (the pre-signature exists only in packed form at the API), so for those operations the boundary behaves like the packed tier; KeyGen, Sign, and Verify exchange parsed in-memory structs, with the wire codecs (`ec_pubkey_parse/serialize`, `ecdsa_signature_parse/serialize`) running *outside* the timed call — the library documents these structs as unsuitable for storage or transmission — so for those three operations the boundary behaves like the core tier.

The classical column is therefore *not* a full native-API tier: it is this hybrid boundary, stated per operation, and the qualification accompanies the classical comparison (table 3.6). Modifying the library to expose uniform boundaries would break the reuse-unmodified posture (section 2.2), so the hybrid boundary is reported as-is instead.

Implementation of record. The scheme exists in two implementations, so each reported measurement states its origin. Unless a caption says otherwise, timing results are measured on the C implementation; the Rust implementation is reported separately in table 3.4. The two are never averaged: they are built by different toolchains (GCC versus rustc/LLVM), so their timings are samples from different populations, and a pooled mean would obscure any systematic disagreement between the codebases. Communication sizes are byte-identical in both implementations by construction (asserted programmatically on every Rust run and pinned by the shared known-answer digest), so size tables apply to both.

Repeatability and machine of record. Every timing is the mean over 5 independent repetitions of 500 iterations per sign-class operation and 1000 iterations per verify-class operation, single-threaded, compiled at `-O3`. Results are reported as mean ± sample standard deviation so that machine and scheduling noise are visible. All non-random inputs are fixed and stated, so that run-to-run variability comes only from rejection sampling and machine noise, never from input selection

[15]: a fixed 33-byte message, a fixed public-parameter seed, and one consistent key/statement state per run, identical between the C and Rust drivers. The *randomised* signing paths are the ones timed — fresh masking randomness is drawn inside every timed call, for the base scheme and LAS alike, so randomness generation sits inside the boundary symmetrically; the deterministic API exists only for the known-answer tests and is never benchmarked. Two validity gates precede every timing: the full correctness contract of section 2.4 is asserted on the very objects about to be timed (so no failure path is ever measured), and the directly counted rejection-loop attempt rate is asserted against its closed-form prediction within five standard deviations (so a broken sampler cannot produce plausible timings). All measurements — C, Rust, and the classical baseline — were taken on one machine: an AMD Ryzen 7 7745HX (8 cores, 16 threads) with about 7.4 GB of RAM visible to the Linux environment, running Ubuntu 24.04.3 LTS (long-term support; kernel 6.6) under the Windows Subsystem for Linux (WSL2) on Windows, with the GNU Compiler Collection (GCC) 13.3.0 and GNU Make 4.3 for C and stable `rustc` in release profile for Rust. The reference implementation, the `fips204` crate, and the classical baseline library are used at fixed versions, and each reported table lists the exact command that produces it (appendix A.1).

Per-operation reporting, not cumulative. Every operation is timed and reported *independently* — KeyGen, Sign, Verify, PreSign, PreVerify, Adapt, and Extract — rather than as a single cumulative end-to-end time. The reason is that the operations are split across parties and machines: in a swap, one party signs or pre-signs, another verifies and later adapts, and a third may extract, so a combined figure would average over costs that are never actually paid together by one party. Per-operation timing also exposes the adaptor overhead operation by operation, which a cumulative number would hide. This is the primary timing result (table A.1); an end-to-end figure adds nothing the per-operation breakdown does not already give.

Primary (fair) comparison: LAS vs. its own simplified base. LAS is defined over a simplified Dilithium-style base signature (Fiat–Shamir with aborts, ternary secrets, no hint vector); LAS is exactly that base plus the four adaptor operations, built on the same code, the same parameters, and the same primitives. The primary comparison therefore holds everything fixed except the adaptor layer,

and measures its *overhead*: this is the only same-algorithm, same-parameter, same-primitive comparison, so any difference it shows is genuinely the cost of adding adaptor functionality. Each adaptor operation is paired with the base operation it mirrors algorithmically:

- PreSign against the base Sign — both sample a masked commitment, hash, and respond under rejection sampling; PreSign only folds the statement Y into the commitment and uses a tighter rejection bound;
- PreVerify against the base Verify — both recompute the same commitment shape, PreVerify with the extra $+Y$;
- Adapt against the base Verify — Adapt runs a PreVerify and then adds the witness;
- Extract is reported separately, having no base-operation analogue.

Rejection sampling: the expected attempt count, derived. The sign-class operations restart until the response passes its norm bound, so their cost is a random multiple of one attempt's cost, and no timing claim about them is meaningful without the attempt statistics. The expectation follows from the construction [6]. Each coefficient of the mask y is uniform on $[-\gamma, \gamma]$, i.e. on $2\gamma+1$ values; the secret-dependent term $c\,r$ satisfies $\|c\,r\|_\infty \le \kappa$ (the challenge has κ nonzero coefficients of ± 1 and the secret is ternary), so each response coefficient $z_i = y_i + (c\,r)_i$ is uniform on a window of $2\gamma+1$ consecutive integers whose centre is offset by at most κ. The acceptance interval $[-(\gamma-\kappa), \gamma-\kappa]$ then always lies inside that window, so each coefficient is accepted with probability exactly $\big(2(\gamma-\kappa)+1\big)/(2\gamma+1)$, *independently of the secret* — which is precisely why rejection sampling makes the response simulatable. An attempt succeeds only if all $(n+\ell)\,d$ coefficients pass:

$$p_{\text{acc}} = \left(\frac{2(\gamma-\kappa)+1}{2\gamma+1}\right)^{(n+\ell)d} \approx \left(1-\frac{\kappa}{\gamma}\right)^{(n+\ell)d} \approx e^{-\kappa(n+\ell)d/\gamma} = e^{-1} \approx 36.8\%, \tag{2.2}$$

where the last step uses the design choice $\gamma = \kappa\,d\,(n+\ell)$ — made in the original construction exactly so that the expected number of restarts is "about $e < 3$" [6]. The attempt count is geometric, so the expected attempts per call are $1/p_{\text{acc}}$: at the target setting, **2.719 for Sign** (bound $\gamma-\kappa$) and **2.775 for PreSign** (the

tighter bound $\gamma - \kappa - 1$, whose window is smaller by two values per coefficient). PreSign is therefore *expected* to take about 2% more attempts than Sign at identical sample sizes — so a PreSign-vs-Sign timing gap of this order is predicted by the bounds themselves, before any adaptor arithmetic is counted, and the measured attempt counters are reported next to the timings so the reader can separate the two effects.

Rejection sampling: the run-validity gate (sample-size argument). Because the attempt count is geometric, per-call timings are heavy-tailed, and averages over too few calls can drift by several percent on rejection luck alone — a documented pitfall of lattice-signature benchmarking [15]. The drivers therefore treat the attempt statistics as a *validity gate* rather than a curiosity: over the n_{calls} timed sign-class calls of a run, the measured mean attempts per call must match the closed-form expectation $E = 1/p_{\text{acc}}$ of eq. (2.2) within five standard errors, $|\bar{A} - E| \le 5\,\sigma/\sqrt{n_{\text{calls}}}$ with $\sigma = E\sqrt{1 - 1/E}$ (the geometric standard deviation), separately for Sign *and* PreSign, at both timing tiers, in C and in Rust; a run that fails the gate aborts instead of producing evidence. Passing the gate is the sample-size argument: it certifies both that the rejection loop behaves exactly as the theory prescribes (a broken sampler cannot produce plausible-looking timings) and that the run is long enough for its mean to sit inside a known, narrow band ($\pm\sim 8\%$ at the C drivers' 2500 calls; $\pm\sim 1\%$ at the Criterion harness's $\approx 10^5$ calls). The exact benchmark procedure, including the gate, is given as pseudocode in appendix A.2.

Parameter sensitivity. To test whether the adaptor overhead depends on the security level, the same base/adaptor comparison is repeated at several parameter sets. The module dimensions for these sets are taken from the standard Dilithium parameter choices at NIST levels 2, 3, and 5 — used here purely as established, well-studied lattice dimensions to scale the scheme across — together with the dimensions of the original LAS construction (table 2.2). Setting the row count n and extra columns ℓ to a level's (K, L), and the challenge weight κ to its τ, fixes the public-key, secret, and challenge dimensions. The implementation's parameters are compile-time constants, so the identical code runs at every set; the bound $\gamma = \kappa\, d\,(n + \ell)$ and the sampler widths follow from them. The concrete bit-security of each set is not formally claimed, as security proofs are beyond the scope of this work.

Table 2.2: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set is the parameter set proposed in [6]; the *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5 (so the public-key, secret, and challenge dimensions line up with NIST levels 2/3/5) and are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table 2.3 defines each symbol.

Setting	n	ℓ	M	κ	γ	d	q
LAS-2020/845 reference	4	4	8	60	122 880	256	8 380 417
Simplified Dilithium-II	4	4	8	39	79 872	256	8 380 417
Simplified Dilithium-III (target)	6	5	11	49	137 984	256	8 380 417
Simplified Dilithium-V	8	7	15	60	230 400	256	8 380 417
Classical (secp256k1 ECDSA adaptor)	≈128-bit classical (see table 3.6)						

Component-level size analysis. Rather than reporting only total object sizes, each key and signature is decomposed into its components (challenge c, response z, statement Y, witness y, pre-signature) so that the *cause* of any size difference can be identified. This breakdown is presented in chapter 3.

Classical baseline (functionality-matched, not security-matched). The second baseline is a classical secp256k1 ECDSA adaptor signature: the `secp256k1-zkp` library's `ecdsa_adaptor` module [2], which implements Fournier's one-time verifiably-encrypted-signature construction [7], used unmodified at a pinned commit. This is *not* a same-security post-quantum comparison: secp256k1 provides approximately 128-bit *classical* security, is not quantum-resistant, and therefore has no corresponding NIST post-quantum level. Instead it is a functionality-matched "price of post-quantum" baseline: both schemes provide adaptor-signature functionality for blockchain-style atomic swaps, but they rely on different hardness assumptions and different algebra (classical discrete-log versus post-quantum lattice). LAS is evaluated as the post-quantum adaptor construction; the ECDSA adaptor represents the compact, widely deployed classical alternative. Plain ECDSA is excluded: the fair counterpart of an adaptor signature is another adaptor signature, not a basic one.

Table 2.3: Security and scheme parameters: the symbol, its meaning, and the value(s) it takes across the parameter sets, *listed in the row order of* table 2.2 (LAS-2020/845 reference; Simplified Dilithium-II; -III; -V). The notation follows the LAS paper [6]: the ring degree is its d, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. The figure and table captions in chapter 3 refer back to these names and values.

Symbol	Meaning	Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V
n	module rank: rows of A, length of public key t	4, 4, 6, 8
ℓ	extra columns of A' (secret-vector extension)	4, 4, 5, 7
$M = n + \ell$	response dimension (number of polynomials in $z, \hat{z}, y$)	8, 8, 11, 15
κ	challenge Hamming weight (number of ± 1 in c; Dilithium τ)	60, 39, 49, 60
γ	rejection / masking bound, $\gamma = \kappa\, d\, (n + \ell)$	122 880, 79 872, 137 984, 230 400
d	ring degree $(X^d + 1)$	256 (all sets)
q	modulus from the reused NTT, $q \approx 2^{23}$	8 380 417 (all sets)

Chapter 3

Results and discussion

This chapter reports what was built and measured, and discusses why the results should be believed. It covers correctness testing, then computation cost, then communication cost with the component breakdown, and finally the application demonstration.

3.1 Correctness and robustness

Before any performance claim, the implementation must be shown to be *correct*. An adaptor signature is not correct merely because it signs and verifies; it must satisfy a specific contract, and a consolidated harness (`test_contract`) checks every clause of that contract and prints each as a labelled pass. Table 3.1 lists the checks and their outcomes.

Each clause matters. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire, clause 2), becomes a basic signature once the witness is added (clause 3), and the act of completing it reveals the witness (clause 4) — the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness: the byte-level verifier an on-chain integration would consume must not trust its input, so it rejects tampering and malformed encodings. Clause 7 is reproducibility, which lets an independent verifier be cross-checked against fixed vectors. All clauses pass, with zero compiler warnings under the project's strict flags.

Table 3.1: The adaptor-signature correctness contract, each clause checked and passed by `test_contract` (corroborated at scale by the 1000-iteration functional suite on modes 2/3/5, the 256-instance serialization suite, and the pinned known-answer test).

#	Property	Result
1	PreSign produces a pre-signature that PreVerify accepts	pass
2	the pre-signature *fails* basic Verify (statement-binding tripwire)	pass
3	Adapt yields a signature that basic Verify accepts	pass
4	Extract recovers the witness *exactly* ($y' = y$)	pass
5a	a tampered message fails Verify	pass
5b	every one of the 4640 single-byte flips of the packed signature is rejected	pass
6	malformed packed bytes are rejected ($pk \geq Q$, ternary code-3, z out of band)	pass
7	the deterministic API is byte-identical on re-run	pass

3.2 Computation cost

The primary computation result is the cost of the *adaptor layer* — what LAS adds to the basic signature it is built on. This is the base-signature path (SIGN and VERIFY, with $c = H(pk, w, M)$ and no statement) against the LAS adaptor path (PRESIGN, PREVERIFY, ADAPT, EXTRACT, with $c = H(pk, w + Y, M)$); the two paths share the same algorithm, parameters, and primitives, so any difference is purely the price of adding adaptor functionality (section 2.6). Each operation is timed *independently*, because in a swap one party signs or pre-signs while another verifies and later adapts, so a single end-to-end figure would be unrealistic.

Table 3.2 is the primary result, and it reports the comparison at *both* measurement boundaries of section 2.6: the core tier, which isolates the adaptor arithmetic, and the packed tier, which includes the validating decode and encode a wire or on-chain consumer pays. Figure 3.1 visualises the core tier: each LAS adaptor operation (orange) is drawn beside the basic operation it mirrors (blue), with the overhead printed above the orange bar, so the headline is read directly off the figure rather than inferred from two distant groups: at the target setting, *Simplified Dilithium-III*, every adaptor operation stays within single digits of its basic counterpart, and Extract — which has no basic analogue — is the cheapest operation of all. Plotting on an absolute μs scale also makes the shape visible — every operation completes in well under two milliseconds, and signing and pre-signing dominate because of rejection sampling, while the verify-class operations

Table 3.2: The primary computation result: per-operation adaptor overhead at the target setting *Simplified Dilithium-III* (n=6, ℓ=5, κ=49, γ=137 984, d=256), at *both* measurement boundaries of section 2.6. Each LAS adaptor operation is paired with the basic operation it mirrors; "Overhead" is the extra cost as a percentage of that basic operation, within the same tier. All timings are measured on the C implementation, the implementation of record (section 2.6); the Rust port's independent measurements are in table 3.4. Timings are μs per operation (mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500/1000 iterations; machine of record in section 2.6). KeyGen, Sign, and Verify are reused unchanged by LAS; Extract has no basic analogue. The packed-tier overheads are larger because each adaptor operation additionally decodes the public-key-sized statement Y (4416 B) — serialization, not adaptor arithmetic. Absolute core-tier timings at every parameter set are in table A.1 (appendix).

Operation (basic / adaptor)	Basic (μs)	LAS adaptor (μs)	Overhead
Core tier (structures in/out — isolates the adaptor arithmetic)			
KeyGen	91.6 ± 2.0	91.6 ± 2.0 (shared)	—
Sign / PreSign	852 ± 25	866 ± 41	+1.6%
Verify / PreVerify	199 ± 6	210 ± 9	+5.1%
Verify / Adapt	199 ± 6	217 ± 5	+9.0%
Extract	—	84.2 ± 6.3	(LAS only)
Packed tier (wire bytes in/out — incl. validating decode + encode)			
KeyGen	314 ± 5	314 ± 5 (shared)	—
Sign / PreSign	1384 ± 37	1673 ± 49	+20.8%
Verify / PreVerify	703 ± 37	951 ± 18	+35.3%
Verify / Adapt	703 ± 37	1261 ± 20	+79.5%
Extract	—	940 ± 43	(LAS only)

and Extract are far cheaper. The same base-versus-adaptor comparison repeated across all four parameter sets is a scaling check, deferred to fig. A.1 (appendix).

The adaptor layer is therefore almost free in computation, and the numbers are small *for structural reasons* (overhead at the target setting against the paired basic operation):

- **PreSign ≈ Sign** (+1.6%): PreSign runs the same masked-commit, hash, respond, reject loop as Sign; it only adds the statement Y into the Fiat–Shamir commitment and rejects at a slightly tighter bound. Neither changes the asymptotic work.

- **PreVerify ≈ Verify** (+5.1%): PreVerify recomputes the same commitment shape as Verify, differing only by the $+Y$ addition before hashing.

- **Adapt ≈ Verify** (+9.0%): Adapt first runs a PreVerify and then adds the witness vector, so it costs about one verification plus a few polynomial additions.
- **Extract is cheapest and has no basic analogue**: it only subtracts $z - \hat{z}$ and checks $A\,y = Y$.

This is the headline of the standalone-signature evaluation: adding adaptor functionality to the basic lattice signature costs only single-digit percentages of extra computation, and the two operations unique to an adaptor signature, Adapt and Extract, cost no more than a verification each. As a robustness check that this is not an artefact of one parameter choice, the same base-versus-adaptor comparison was repeated at four parameter sets (the LAS-paper dimensions and the Simplified Dilithium-II/III/V dimensions); the adaptor premium stays in the single digits at every one (fig. A.1, appendix).

The structural argument above concerns the core tier. The packed-tier block of table 3.2 shows what changes when the measurement boundary moves to packed bytes, as a wire or on-chain verifier must operate: the adaptor premium grows to +20.8% for PreSign, +35.3% for PreVerify, and +79.5% for Adapt. This gap is not adaptor arithmetic but serialization: each adaptor operation must additionally decode the pk-sized statement Y (4416 B), and Adapt also re-packs the completed signature. The object that dominates the *communication* cost (section 3.4) therefore reappears as the dominant *computation* cost once the byte boundary is included, so a deployment that verifies from bytes should budget for the packed-tier row rather than the core one. Presenting the two tiers side by side, with the boundary of each stated, is deliberate: reported lattice-signature timings are notoriously sensitive to where the measurement boundary is drawn [15].

Signing and pre-signing are dearer than verification because they use rejection sampling, and the attempt counters measured during the timed calls explain the Sign-versus-PreSign timing gap; table 3.3 collects the statistics and fig. 3.2 shows the distribution they come from. The closed-form expectation of eq. (2.2) predicts 2.719 attempts per Sign and 2.775 per PreSign at the target setting — PreSign is expected to restart about 2% more often, because its bound is tighter by exactly one. Measured directly, by counting rejection-loop attempts rather than estimating from a timing ratio, the run of record observed 2.715 attempts per Sign (36.8% acceptance) and 2.716 per PreSign (36.8%): both sit comfortably inside the five-standard-error gate of section 2.6, which every run

Table 3.3: Rejection-sampling statistics at the target setting *Simplified Dilithium-III*: the closed-form geometric model of eq. (2.2) against every measured sample. "Mean" is attempts per call and "Accept." is the per-attempt acceptance rate — the two quantities every source records, and the ones the model predicts. The per-call *dispersion* (the geometric standard deviation $\sigma = E\sqrt{1 - 1/E}$, the medians and 95th percentiles, and the sampled maxima) is shown in the companion distribution of fig. 3.2 rather than repeated here. The C driver is the implementation of record; the Rust protocol driver and the Criterion harness are the independent samples of section 2.5. Every measured row passes the five-standard-error validity gate of section 2.6.

Operation	Source (calls)	Mean	Accept.
Sign	geometric model, eq. (2.2)	2.719	36.8%
	C driver, distribution sample (2000)	2.715	36.8%
	C driver, timed-run gate (2500)	2.677	37.4%
	Rust driver, timed-run gate (2500)	2.677	37.4%
	Rust Criterion, gate (94395)	2.725	36.7%
PreSign	geometric model, eq. (2.2)	2.775	36.0%
	C driver, distribution sample (2000)	2.716	36.8%
	C driver, timed-run gate (2500)	2.714	36.9%
	Rust driver, timed-run gate (2500)	2.765	36.2%
	Rust Criterion, gate (94395)	2.777	36.0%

must pass before any timing is reported, so a broken sampler cannot silently produce plausible-looking numbers. At the drivers' 2500 timed calls the gate band ($\pm\sim 8\%$) is wider than the 2% theoretical separation, so the two measured counts are statistically indistinguishable at this sample size — which is exactly why the core-tier PreSign-versus-Sign overhead (+1.6%) should be read as "of the same order as the attempt-count prediction", not as a precise constant; the $\approx 10^5$-call Criterion harness, whose band is $\pm\sim 1\%$, resolves the separation (measured 2.725 per Sign against its 2.719 prediction, 2.777 per PreSign against 2.775). Rejection sampling is intrinsic to the Fiat–Shamir-with-aborts family, not a defect of this implementation.

3.3 Cross-implementation check: C implementation versus Rust port

The scheme exists twice, in C and in Rust (section 2.5), and the two implementations agree at every level at which they can be compared. At the byte level the agreement is exact: both produce the same packed sizes at the target setting (public key 4416 bytes, signature 6720 bytes, checked programmatically on every Rust run) and the same pinned known-answer digest (`bb6ad0da...260c`), a single SHAKE256 hash folded over every object of four fully deterministic test vectors. Since the digest covers key generation, signing, the four adaptor operations, and the serialization, a divergence anywhere in either implementation's sampling, algebra, or encoding would flip it; its equality is therefore a strong, cheap argument that the two codebases compute the same scheme.

Timing agreement is necessarily looser, since the two builds go through different compilers (GCC versus rustc/LLVM), but it is close: table 3.4 shows every operation agreeing within about 17% (the largest gap is KeyGen), with the Rust port slightly faster on the verify-class operations and slightly slower on key generation. More importantly, the central conclusion — that the adaptor layer is cheap — reproduces in both languages. The Rust port measures the per-operation overhead at +1.1% (PreSign vs Sign) and +0.5% (PreVerify vs Verify), and −4.9% for Adapt vs Verify, all single-digit in magnitude alongside the small C values of +1.6%, +5.1%, and +9.0%. The Rust Adapt figure is slightly *negative*: at this scale the genuine adaptor overhead is smaller than the cost gap between the base and LAS verify paths, which are separate (duplicated) helpers that compile

Table 3.4: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* (μs per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions $\times$ 500/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

Operation	C implementation (μs)	Rust port (μs)	Rust / C	Rust, Criterion (μs, 95% CI)
KeyGen	91.6 ± 2.0	108 ± 8	1.17	84.4 [83.8, 85.0]
Sign	852 ± 25	894 ± 68	1.05	751 [745, 757]
Verify	199 ± 6	203 ± 18	1.02	152 [151, 153]
PreSign	866 ± 41	903 ± 28	1.04	773 [767, 779]
PreVerify	210 ± 9	204 ± 17	0.97	153 [152, 154]
Adapt	217 ± 5	193 ± 10	0.89	156 [155, 158]
Extract	84.2 ± 6.3	71.7 ± 1.3	0.85	57.4 [56.9, 58.0]

to marginally different code, so the measurement floor — not a real speed-up — fixes its sign. The packed tier reproduces as well: at the packed boundary the Rust port measures +1.2% for PreSign, +12.2% for PreVerify, and +27.1% for Adapt — all positive and in the same order as the C packed-tier figures (Adapt > PreVerify > PreSign), confirming that once the statement-decoding cost is included the adaptor premium is a robust cross-language signal, not a compiler artefact. Both ports' directly counted rejection rates (2.68 attempts per signature, 2.77 per pre-signature) pass the same five-standard-deviation gate against the closed-form prediction. The Criterion harness, with a wholly independent measurement protocol, gives lower absolute times but the same ordering and the same sign-class-versus-verify-class shape; fig. 3.3 shows its raw output for PreSign, the sampling distribution behind one cell of table 3.4. The headline results of this chapter are therefore properties of the algorithm, not artefacts of one codebase, one compiler, or one timing harness.

Table 3.5: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting, in packed wire bytes (not on-chain gas), with its paper notation. "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. Three facts read off directly: the signature, pre-signature, and adapted signature are byte-identical (adaptation changes the *value* of the response, not its size); the response z dominates every signature; and the only extra *public* object LAS adds over a basic signature is the statement Y, the size of a public key. The sizes hold for the C and the Rust implementation alike: the wire encoding is byte-identical in both by construction, asserted programmatically on every Rust run (section 3.3). Sizes at every parameter set: table A.2 (appendix).

Object	Notation	Bytes	In the basic signature?
Public key	$pk = t$	4416	yes (shared)
Secret key	$sk = r$	704	yes (shared)
Challenge	c	32	yes
Response	z / $\hat{z}$	6688	yes
Signature	(c, z)	6720	yes (basic signature)
Statement	$Y = t'$	4416	LAS only (public)
Witness	$y = r'$	704	LAS only (private)
Pre-signature	$(c, \hat{z})$	6720	LAS only
Adapted signature	(c, z)	6720	LAS (verifies as basic sig)

3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap. The size results show where the real cost lies. Table 3.5 decomposes every LAS object into bytes at the target setting; the aim is to explain *which* component drives the size, not merely to state that signatures are large. Communication sizes are fixed by the encoding, so they are reported once, as a table of exact bytes with no dispersion (section 2.6); the same decomposition at every parameter set is tabulated in table A.2 (appendix).

Two points follow, and together they give the sharp conclusion of the evaluation (table 3.5):

- **The response z is essentially the entire signature** — 99.3% at Simplified Dilithium-II, rising to 99.7% at Simplified Dilithium-V. The challenge is carried as the fixed 32-byte commitment hash $\tilde{c}$ (the FIPS 204 encoding [13]; section 2.3) and is irrelevant to size. Any optimisation of LAS signatures is an optimisation of z.

- **An adaptor swap additionally transmits a public-key-sized statement Y and a signature-sized pre-signature**, while the witness is small. These are the only objects an adaptor protocol sends that a basic signature does not.

The sharp conclusion: the cost of LAS is not adaptor computation but communication — specifically the response vector z and the statement Y. The computation results showed the adaptor operations add at most +10.5% of compute over the basic scheme; this section shows the objects that move on the wire are one to a few kilobytes, dominated by z. For a blockchain setting, where every byte is stored and replicated, this is the property that matters, and it is a far more useful statement than "the signature is 6720 bytes".

The response is stored at its full width: each coefficient occupies 18–19 bits (table A.2) rather than being range-reduced or entropy-coded. This is the clearest opportunity for reducing the on-wire footprint, since compressing z would shrink the signature, pre-signature, and settlement payload together; it is left as future work (section 5.2).

3.4.1 The price of post-quantum: classical adaptor baseline

The second baseline is a classical secp256k1 ECDSA adaptor signature [2]. It is the natural functional counterpart of LAS: both are adaptor signatures with the same operation set (KeyGen, Sign, Verify, PreSign, PreVerify, Adapt, Extract) and both drive scriptless atomic swaps. They are compared on equal footing functionally, while differing in their security foundation: secp256k1 provides approximately 128-bit *classical* security and is not quantum-resistant, whereas LAS rests on lattice assumptions believed to resist quantum attack. The comparison therefore measures the practical cost of moving from a classical adaptor signature to a post-quantum one. Plain ECDSA is not used, because the counterpart of an adaptor signature is another adaptor signature.

For this baseline LAS is instantiated at the *Simplified Dilithium-II* parameters, and only that setting is used. The reason is that secp256k1 offers roughly 128-bit classical security, and the nearest post-quantum target is NIST level 2 (also pitched at the 128-bit category); aligning LAS to the Dilithium-II dimensions therefore gives the most like-for-like security target for the classical scheme. Comparing the 128-bit classical adaptor against LAS at the Simplified Dilithium-III or -V

settings would pit it against a deliberately stronger post-quantum configuration and would understate the classical scheme, so the higher settings are reserved for the parameter-sensitivity study of section 3.2 and not used here. Table 3.6 reports both schemes at their adaptor operations. Following the tier rule of section 2.6, LAS appears at both of its measurement boundaries, while the classical scheme is measured at the *hybrid native-API boundary* defined there — not a uniform tier. Per operation: PreSign, PreVerify, Adapt, and Extract include the 162-byte pre-signature's serialization or parsing *inside* the timed call (packed-like), whereas KeyGen, Sign, and Verify exchange in-memory structs with their wire codecs outside the timed call (core-like). The comparison is therefore read conservatively: against LAS's *core* tier it slightly flatters LAS on the adaptor operations (the classical numbers carry pre-signature codec work the LAS core numbers do not), and against LAS's *packed* tier it flatters the classical scheme. Either way the conclusions below survive, because they rest on order-of-magnitude gaps, not on percent-level differences.

This baseline compares two adaptor-signature schemes with the same high-level functionality but different security assumptions: classical discrete-log security versus post-quantum lattice security. Two findings stand out. First, the post-quantum price is paid almost entirely in *size*: LAS's signature and public key are roughly 72× and 89× larger than the classical adaptor's, whereas in computation every LAS operation stays well under a millisecond — even the largest per-operation factor (Adapt, about 52×, against a classical operation that is nearly free) costs under a tenth of a millisecond in absolute terms. Second, the two schemes pay their *adaptor* overhead in opposite ways. The classical ECDSA adaptor must attach a discrete-logarithm-equality proof [7], so its PreSign and PreVerify are several times its own basic sign and verify; LAS folds the statement into a hash it already computes, so its adaptor operations stay within a few percent of its base operations. In computation, then, the lattice adaptor layer is relatively cheaper to add than the classical one; the post-quantum cost surfaces in communication, consistent with the component analysis above.

3.5 Application demonstration

The implementation was demonstrated in an end-to-end post-quantum atomic swap that follows the protocol of [6] (their Fig. 1) *verbatim*, including its proof of

knowledge π. Two parties, Alice and Bob, swap coins across two ledgers with no trusted third party and no on-chain scripts. Alice, the witness holder, commits first: she draws a fresh statement/witness pair (Y, y), produces a zero-knowledge proof π that Y has a *ternary* preimage $(\exists\, r' : \mathbf{A}r' = Y \wedge \|r'\|_\infty \leq 1)$, pre-signs the transaction spending her own coin, and sends $\{Y, \pi, \hat{\sigma}_1\}$ in one off-chain message. Bob pre-signs his own transaction against the *same* Y only after both π and Alice's pre-signature verify — the protocol's abort gate; at this point neither pre-signature is spendable. Alice, who knows y, adapts and publishes her signature to claim Bob's coin; the act of publishing reveals y, which Bob extracts and uses to adapt and publish his own signature, claiming Alice's coin. Either both legs settle or neither does. The proof π is what makes Bob's final step *guaranteed* rather than merely expected: by the Module-SIS uniqueness argument of [6], the extracted witness equals the proven ternary one, so Bob's adapted signature is certain to clear the verification bound — without π, a malicious Alice could claim Bob's coin and leave him with an unadaptable pre-signature. The demo prints this narrative and asserts every step, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement.

The proof π is realised by reusing the LaZer lattice zero-knowledge library [10] rather than implementing a proof system from scratch — the same reuse posture applied throughout. Because LaZer proves per-partition binary coefficients or exact Euclidean norms, the ternary statement is encoded by binary decomposition, $[\mathbf{A} \mid -\mathbf{A}]\,(r_+ \,\|\, r_-) = Y$ with both halves proven binary, which is equivalent to knowledge of a ternary preimage $r' = r_+ - r_-$. The generated parameter set reports a knowledge error below 2^{-127} under Module-SIS and a proof size of approximately 31 kilobytes; proving and verifying each complete in well under a second. The proof is exchanged off-chain only, exactly as [6] note, so it adds nothing to the on-chain cost story. A dedicated test suite additionally asserts completeness, rejection of every sampled single-byte tamper, rejection against a wrong statement, and that the prover refuses a non-ternary witness.

A small ledger abstraction takes this closer to a real chain: accounts with balances, a block height, and adaptor-locked contracts (the scriptless analogue of a hash-time-locked contract, with the hash lock replaced by the statement Y and the time lock by a timeout height). Three scenarios are asserted: a cross-chain swap (happy path), a timeout/refund path in which a premature refund is rejected

and both parties recover their funds after the timeout, and a multi-hop payment.

As a supporting check on deployability, native on-chain LAS verification was *implemented* and measured on a real Solidity stack, at the target Simplified Dilithium-III parameter set. A complete, numerically-correct byte-level verifier (`LASVerify.verify`) was assembled from vendored ZKNox ETHDILITHIUM primitives — SHAKE256, the NTT, and SampleInBall — and validated end-to-end against the C reference: it accepts the exported adapted signature and rejects tampered bytes. (The ZKNox library is an experimental, self-described non-production research artefact, reused here only to price and validate the verifier, not as deployable code.) Wired into the swap as `claimLASVerified`, with the public parameters A', the public key, and the message bound to the escrow by a commitment fixed at funding time, one full verified settlement was measured at approximately 56.5 million gas (table 3.7; fig. 3.4) — roughly 746× the gas of settling the equivalent classical ECDSA-adaptor claim (which verifies in a native precompile, whereas LAS runs entirely in EVM bytecode). This supersedes an earlier op-count *estimate* of approximately 16.7 million gas (13.93 million measured arithmetic plus a 2.76-million modelled hash); the measured figure is larger precisely because a complete verifier must also unpack and decode the signature, run a real Solidity SHAKE256, and canonically pack its inputs — costs the earlier estimate had flagged as omitted. At approximately 56.5 million gas the verifier exceeds the EIP-7825 per-transaction cap. Its raw execution demand is below the 60-million default block gas limit, but it remains unexecutable because the independent per-transaction cap is 16,777,216 gas. This on-chain result — for this D3 LAS instance in Solidity — and the multi-hop variant are supporting evidence; they inform, but do not displace, the first-stage signature results above.

3.6 Threats to validity

Several factors qualify the results, and are stated here so the reader can judge how far to trust them. First, all timings are machine-dependent; this is mitigated by reporting the standard deviation, fixing one machine and compiler, emphasising ratios over absolute values, and by the cross-implementation check (section 3.3), which reproduces every relative conclusion under a different compiler and an independent timing harness. Second, the LAS parameter set's concrete security level is not formally established, because security proofs are out of the project's

scope; every cross-scheme size and speed claim is therefore qualified by the explicit parameters in table 2.2, and the fair comparison deliberately holds parameters fixed between LAS and its simplified-Dilithium base. Third, the modulus is $q \approx 2^{23}$, inherited from the reused NTT table, rather than the 2^{24} specified in [6]; correctness is unaffected ($q > 2\gamma$) but the security margin differs. Fourth, the extracted witness is exact in this parameterisation, whereas the relaxed setting of [6] allows witness noise that can grow across long payment-channel paths; the exact-extraction choice sidesteps that growth for the demonstration but is a simplification worth noting.

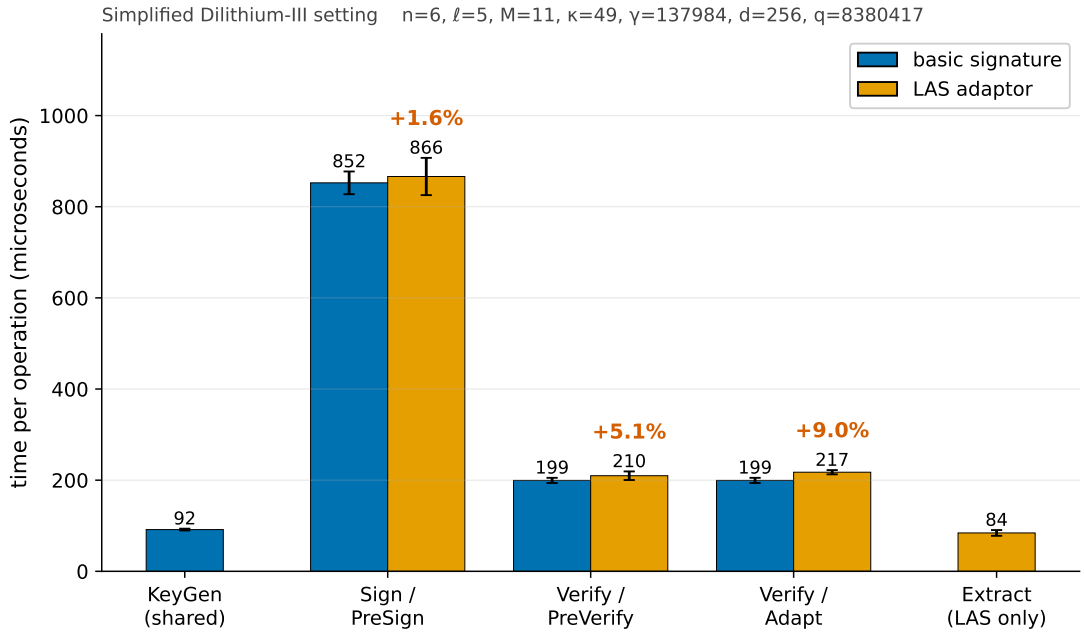

Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot), measured on the C implementation (the implementation of record, section 2.6). The percentage above each orange bar is the adaptor overhead over its blue partner; exact means and standard deviations, and the packed tier, are in table 3.2. KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. The absolute μs scale also shows the shape: every operation is sub-two-milliseconds and the rejection-sampling operations (Sign, PreSign) dominate, while the verify-class operations and Extract are far cheaper. Error bars are the sample standard deviation over 5 repetitions of 500 (sign-class) / 1000 (verify-class) iterations on the machine of record (section 2.6); the same comparison across all four parameter sets is the scaling check in fig. A.1 (appendix).

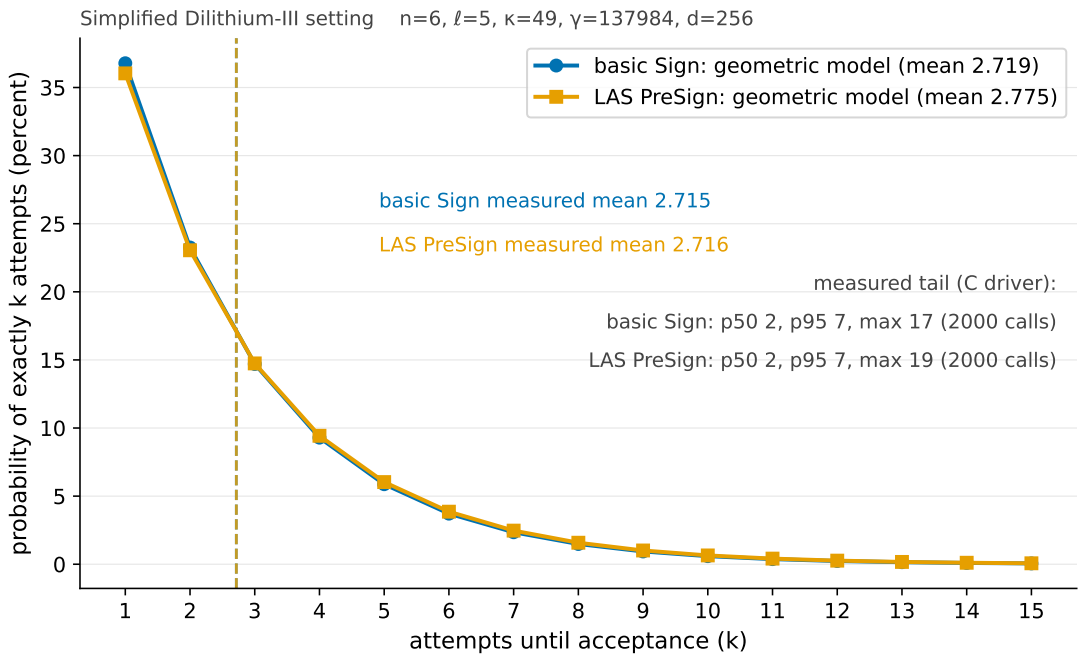

Figure 3.2: The rejection-attempt distribution at the target setting *Simplified Dilithium-III*. The curves are the closed-form geometric model of eq. (2.2) for the basic Sign bound $\gamma-\kappa$ (blue) and the tighter LAS PreSign bound $\gamma-\kappa-1$ (orange) — nearly coincident, which is exactly why the PreSign-versus-Sign timing gap is so small. The dashed vertical lines and the annotated statistics are *measured* on the C implementation (2000-call distribution sample of table 3.3): means 2.715/2.716, medians 2, 95th percentiles 7, and observed maxima of 17 (Sign) and 19 (PreSign) attempts — the heavy geometric tail that makes per-call signing times disperse.

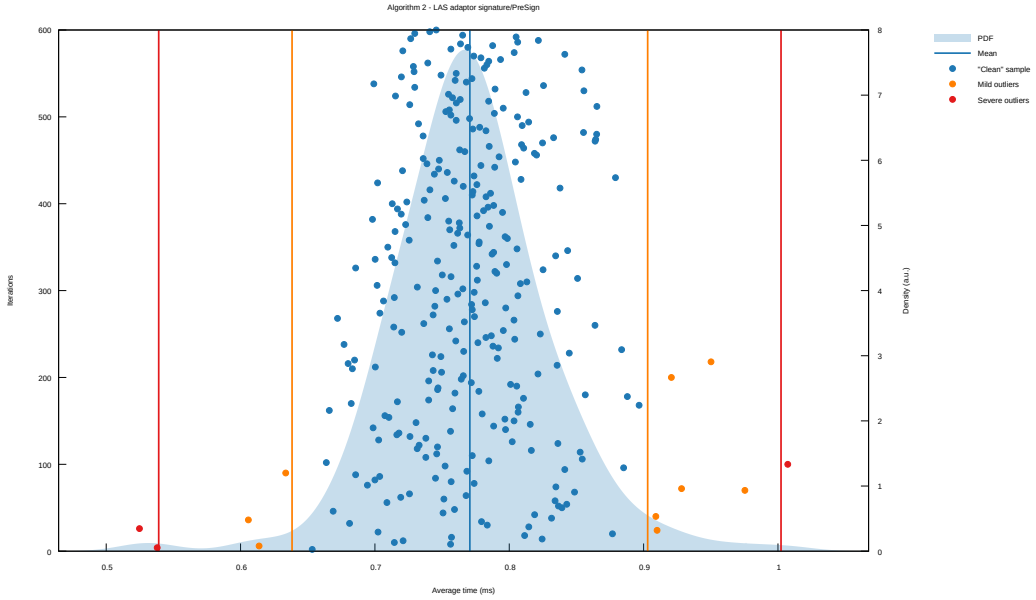

Figure 3.3: The statistical harness's own report for PreSign at the target setting, reproduced unmodified from Criterion.rs (its SVG output converted to PDF): 300 timed samples (dots), the kernel-density estimate of the sampling distribution (shaded), the mean (vertical blue line), and Criterion's automatic outlier classification (mild / severe, orange / red). This is the evidence behind the Criterion column of table 3.4: a well-formed, single-mode distribution whose spread comes from rejection sampling and scheduling noise, with only a handful of classified outliers.

Table 3.6: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) versus LAS at the *Simplified Dilithium-II* parameters ($n = 4$, $\ell = 4$, $M = 8$, $\kappa = 39$, $\gamma = 79\,872$, $d = 256$, $q = 8\,380\,417$). The Dilithium-II dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim (table 2.2); the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings μs/op (classical: mean ± SD over 10 runs × 1000 iterations; LAS: 5 repetitions × 500/1000 iterations; same machine); sizes in bytes. Measurement boundaries (section 2.6): the classical column is the library's *hybrid* native-API boundary — PreSign/PreVerify/Adapt/Extract pack or parse the 162-byte pre-signature *inside* the timed calls (packed-like), while KeyGen/Sign/Verify exchange in-memory structs with their wire codecs outside (core-like); LAS is shown at both its core and packed tiers, measured on the C implementation. KeyGen also produces the statement/witness, which take the public-key/secret-key size. The classical pre-signature is a syntactically distinct object (an ECDSA signature plus a discrete-logarithm-equality proof), whereas the LAS pre-signature shares the signature format.

	ECDSA adaptor (classical, hybrid native API)	LAS adaptor (post-quantum, core tier)	LAS adaptor (post-quantum, packed tier)
Computation (μs/op)			
KeyGen	27.2 ± 1.9	61.7 ± 0.8	221 ± 15
Sign	36.1 ± 1.0	570 ± 20	893 ± 27
Verify	58.1 ± 7.8	131 ± 3	326 ± 45
PreSign	171 ± 10	608 ± 35	1136 ± 56
PreVerify	219 ± 7	135 ± 2	590 ± 14
Adapt	2.8 ± 0.1	144 ± 7	835 ± 19
Extract	30.6 ± 2.5	55.6 ± 4.2	641 ± 12
Communication (bytes)			
Public key / statement	33	2944	
Secret key / witness	32	512	
Signature	64	4640	
Pre-signature	162	4640	

Table 3.7: On-chain settlement gas for the atomic swap's claim step, measured on a local EVM (Foundry `forge -gas-report`). Gas is deterministic for the fixed bytecode, EVM revision, inputs, and contract state, and is independent of the host CPU. The ratios compare complete settlement calls, including calldata, memory, state transition, event emission, transfer, and verification. Both the classical path and the full LAS path perform *complete* signature verification — the difference is that ECDSA verifies in the native `ecrecover` precompile whereas LAS runs its entire verifier in EVM bytecode. `claimLAS` is a floor that performs *no* lattice verification (calldata for the 6720-byte signature plus one `keccak256`), a strict lower bound. `claimLASVerified` runs the complete native `base_verify` (section 2.1) — decode, the norm bound, $w' = A\mathbf{z} - c\,t$, and the SHAKE256 challenge check — assembled from the vendored ZKNox ETHDILITHIUM primitives and validated end-to-end against the C reference. At ≈56.5 M gas it exceeds the EIP-7825 per-transaction cap ($16\,777\,216 = 2^{24}$) by ≈3.4×, so it cannot execute as a single mainnet transaction; its raw demand is below the 60-million default block gas limit, but the per-transaction cap is the binding ceiling.

Claim path	On-chain verification	Gas	× classical
Classical ECDSA adaptor (`claimClassical`)	full, `ecrecover` precompile	75 751	1×
LAS floor (`claimLAS`)	none (calldata + `keccak256`)	289 930	3.8×
LAS full verify (`claimLASVerified`)	full `base_verify`, in Solidity	56 538 682	746×

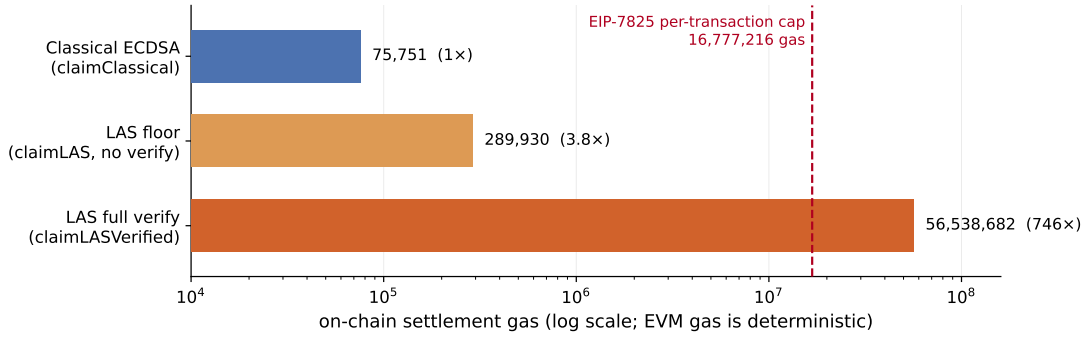

Figure 3.4: On-chain settlement gas for the three atomic-swap claim paths (table 3.7), measured on a local EVM (`forge -gas-report`; deterministic EVM gas), on a logarithmic axis. The classical `claimClassical` (75 751 gas) verifies in the native `ecrecover` precompile; `claimLAS` (289 930) is a settlement floor doing *no* lattice verification; and `claimLASVerified` (56 538 682, ≈746× the classical claim) runs the complete native LAS `base_verify` in EVM bytecode. The dashed line is the EIP-7825 per-transaction gas cap ($16\,777\,216 = 2^{24}$); the full verifier's bar crosses it, which is exactly why it cannot execute as a single mainnet transaction and why a deployable path needs a precompile, a succinct proof, or an optimistic (Naysayer) scheme.

Chapter 4

Evaluation and reflection

This chapter evaluates the work against the objectives set out in section 1.3 and reflects on the soundness of the approach, building on the validity discussion in section 3.6.

4.1 Evaluation against the objectives

Each objective from section 1.3 is judged in turn against the evidence in chapter 3.

1. *Literature and scheme selection — met.* The adaptor signature was selected as a representative exotic scheme with high blockchain value, and LAS [6] as the design with the clearest path from a standardised lattice signature, following the survey's framing of the gap [3] and the supervisor's steer toward the simplest viable construction.

2. *Additive implementation — met.* The lattice core is reused byte-for-byte (table 2.1); zero upstream functions were modified, which the two-branch diff makes auditable. The adaptor layer, wire encoding, and application are new code.

3. *Validation — met.* Functional, serialization, tamper, and known-answer tests all pass (section 3.1), establishing the adaptor contract, wire-encoding integrity, and reproducibility. Beyond the single-codebase tests, an independent Rust implementation reproduces the wire format and the pinned known-answer digest byte-for-byte (section 3.3), the strongest correctness evidence available short of a formal proof.

4. *Fair benchmarking — met.* The primary comparison holds algorithm, parameters, and primitives fixed between LAS and its own simplified base, so the primary comparison isolates the adaptor overhead — the base-signature path against the LAS adaptor path, every operation timed independently. The adaptor overhead is reported per operation as the headline (table 3.2), at *both* declared measurement boundaries — the core tier and the packed tier — so no reported speed reflects an undocumented benchmarking scope (section 2.6); the absolute timings and a parameter-sensitivity check across four settings are shown as supporting context (figs. 3.1 and A.1, exact mean ± standard-deviation figures in table A.1); computation and communication are separated and broken down by component. The optimised CRYSTALS-Dilithium reference is included only as context (not algorithm-matched), and a classical ECDSA adaptor as a functionality-matched "price of post-quantum" baseline, each with the standard five metrics. The measurements themselves are defended two ways: every run gates on a directly counted rejection rate matching its closed-form prediction, and the Rust port re-measures the same operations under a different compiler and an independent statistical harness, reproducing every relative conclusion (table 3.4).

5. *Atomic-swap demonstration — met at the simulated-ledger level.* An end-to-end two-party, two-chain swap settles both legs and asserts unspendability before adaptation (section 3.5). A full deployment on a live testnet was not in scope; instead, on a local EVM both an on-chain settlement floor and a *complete*, validated native LAS verification are measured (section 3.5; table 3.7), the latter at approximately 56.5 million gas — above the EIP-7825 per-transaction gas cap, and thus concrete evidence for why deployment needs a precompile, a succinct proof, or an optimistic verification scheme.

4.2 Challenges encountered during the modification

Turning the existing simplified-Dilithium base into LAS was conceptually a small change — four added operations and one extra term in a hash — but four challenges consumed most of the engineering effort, and a fifth arose later in

taking the verifier on-chain. Stating them explicitly is part of explaining how the results were produced.

1. **The rejection-bound budget.** The single most delicate line of the scheme is PreSign's bound $\gamma - \kappa - 1$ (eq. (2.1)). Setting it one looser, at the base scheme's $\gamma - \kappa$, produces an implementation that passes every local test — pre-signatures pre-verify, extraction works — yet whose *adapted* signatures can exceed the basic verifier's bound and fail. The failure is silent, probabilistic, and appears two operations away from its cause, so it had to be prevented by reasoning through the norm budget ($\|y\|_\infty \leq 1$, hence $\|\hat{z} + y\|_\infty \leq \gamma - \kappa$) rather than discovered by testing.

2. **The reference's optimisations fight the adaptor identity.** The adaptor mechanism rests on the exact identity $A\mathbf{z} - c\,t = w + Y$. The reference implementation deliberately breaks exactness for compactness — Power2Round, the hint vector, and high/low-bit decomposition all discard low-order information. Understanding *which* optimisations had to be disabled, and why the identity fails with each of them enabled, was a prerequisite to a working PreVerify at all; it is also why the simplified scheme is a design requirement rather than a convenience (section 2.2).

3. **Living inside another scheme's namespace.** The reused reference globally defines single-letter parameter macros — its `N` is the ring degree and its `K` the module rank — while LAS needs its own, different dimensions with the same natural names. An innocently named LAS constant silently inherits the wrong value through the preprocessor instead of failing to compile. This forced a strict naming discipline (prefixed LAS parameters, a documented C–Rust–paper name map) and explains some otherwise-surprising names in the code.

4. **Benchmarking a variable-time scheme fairly.** Rejection sampling makes sign-class timings heavy-tailed, and two early artefacts showed how easily this misleads: an acceptance rate first estimated from a timing ratio was biased (it implied $\approx$23% where direct counting gives 36.8%), and the two languages' base signing paths initially used different norm-check strategies (full-scan versus early-exit) *inside* the timed loop — functionally identical, KAT-invisible, but a work-profile asymmetry that biased the primary overhead comparison. Both lessons are now institutionalised in the methodology:

attempts are counted directly and gated against theory (section 2.6), the drivers mirror each other line-for-line, and every measurement boundary is declared.

5. **Verifying on-chain, and why the reference is the only arbiter.** Porting `base_verify` to Solidity to *measure* the on-chain gas (section 3.5) surfaced a trap sharper than any arithmetic bug. The public matrix A' is stored by the reference *already in the NTT domain* — its uniform sampler emits the transformed matrix directly — and a first verifier transformed it a second time. The error was *self-consistent*: the recomputed commitment w' and the recomputed challenge hash agreed *with each other*, so every primitive checked in isolation passed, yet the whole disagreed with the C reference, and was found only by an end-to-end check that recomputes the challenge digest against the reference's. The lesson shaped the port's whole testing method: for a cryptographic re-implementation, a shared cross-implementation golden value is the only reliable arbiter and internal self-consistency is *not* correctness, which is why the verifier was validated in stages against vectors exported from the C code — each primitive (SHAKE256, the NTT, SampleInBall, the response decode), then the assembled verifier, must match — rather than merely made to run. A second, related mismatch — the reference NTT is Montgomery-domain whereas the reused third-party transform is normal-domain, so the two cannot share transformed data — had to be resolved by recovering the normal-domain matrix before reuse. One caveat belongs with the figure: those reused primitives come from an experimental, self-described non-production library, so the reported gas is a faithful cost of *a* numerically-correct verifier, not an endorsement of that library for deployment.

A sixth, cross-cutting difficulty — reproducing the C scheme byte-for-byte in Rust, down to sampler subtleties such as how each language's uniform sampler discards rejected blocks — is discussed with the cross-validation results (section 3.3); the pinned known-answer digest was the tool that turned that difficulty into a mechanical, checkable process.

4.3 Reflection on the approach

Five reflections stand out. First, reuse was the right strategy: building additively on a standardised reference made the implementation tractable in the available time and produced an unusually strong trust story, since the security-critical arithmetic is exactly the audited upstream code. Second, the deepest challenges (section 4.2) were not the adaptor mathematics but its *interactions* — with the base scheme's norm budget, with the reference's optimisations, and with its namespace — which is a useful general lesson about modifying cryptographic reference code. Third, measuring the rejection rate directly, by counting attempts, corrected an earlier figure that had been estimated from a biased timing ratio — a concrete lesson in preferring direct measurement to a proxy; that lesson is now institutionalised as a gate every benchmark run must pass. Fourth, the second implementation repaid its cost: because the reuse strategy transfers across languages, the Rust port was comparatively cheap to build, yet it upgraded the correctness argument from "one tested codebase" to "two independent codebases that agree byte-for-byte" and exposed the timing conclusions to an independent harness. Finally, confronting the fairness problem honestly, by stating every scheme's parameters and comparing LAS to a parameter-matched base, strengthens the conclusions rather than weakening them: the size gap to optimised Dilithium is shown to be a packing gap, not a dimension gap.

4.4 Limitations

The limitations follow from the validity discussion in section 3.6 and are restated compactly here. There is no formal security analysis of the chosen parameter set, by design. The modulus is $q \approx 2^{23}$ rather than the 2^{24} specified in [6]. The witness is extracted exactly, sidestepping the relaxed, noisy-witness setting of [6] and the witness growth it implies for long channels. And the simplified, hint-free scheme produces larger keys and signatures than optimised Dilithium. Each of these is a deliberate scope choice that trades cryptographic optimisation for a transparent, testable artefact, and each maps directly to an item of future work in section 5.2.

Chapter 5

Conclusion and future work

5.1 Conclusions

This dissertation set out to implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a blockchain atomic swap. That aim was achieved. A complete adaptor signature was built additively on the Dilithium reference, with the lattice core reused unchanged and the adaptor layer, wire encoding, and application added as new, tested code. The scheme passes functional, serialization, tamper, and known-answer tests, is corroborated by an independent Rust implementation that reproduces the wire format and the pinned known-answer digest byte-for-byte, and drives an end-to-end two-party, two-chain atomic swap.

The evaluation answers the project's central question — what does adaptor functionality cost over a plain signature, and what does post-quantum operation cost over a classical one — with measured data, at explicitly declared measurement boundaries. At the core boundary, which isolates the adaptor arithmetic, the primary fair comparison of LAS against its own simplified base shows the adaptor layer is almost free in computation: PreSign, PreVerify, and Adapt each add only single-digit overhead (at most +10.5%) over the basic operation they mirror, at every parameter set — a conclusion both implementations reproduce independently — and this is relatively cheaper than the classical ECDSA adaptor, whose discrete-logarithm-equality proof dominates its adaptor cost. At the packed boundary, where every input is decoded from and every output encoded to wire bytes, the adaptor premium rises to +20.8–79.5% — not because the adaptor arithmetic

grows, but because each adaptor operation must additionally decode the public-key-sized statement Y. The real cost is therefore communication, and it is visible on both axes: the signature is 99.3–99.7% lattice response vector, an adaptor swap additionally transmits a public-key-sized statement, and that same statement dominates the byte-boundary compute cost. The contextual comparison against the optimised CRYSTALS-Dilithium reference — not algorithm-matched — shows that the remaining size gap is one of packing and production optimisation, not of security level.

Measured against the objectives, all five were met: the scheme was selected and justified, implemented by reuse, validated, benchmarked fairly against two baselines with the standard metrics, and demonstrated in an application. The principal qualification is scope: the parameter set's concrete security is not formally analysed, and the application runs on a simulated ledger rather than a live testnet.

5.2 Future work

The results point to several well-motivated directions.

- **Tighter packing.** Apply Dilithium-style decomposition and a hint vector to the response, and regenerate the public key from a seed, to close most of the size gap identified in section 3.4 — the single largest practical improvement available.
- **Parameter migration.** Move to the $q \approx 2^{24}$ of [6] with a new NTT table, and map the parameters to NIST security levels, reporting before-and-after benchmarks so the security/cost trade-off is explicit.
- **On-chain verification.** A complete, numerically-correct native Solidity LAS verifier is now implemented and measured (section 3.5; approximately 56.5 million gas, above the EIP-7825 per-transaction cap of 16,777,216). The remaining work is to bring that within a single transaction — via a dedicated precompile, a succinct (zero-knowledge) proof of verification, or an optimistic (Naysayer) scheme — and to benchmark a swap on a live testnet.
- **A new research question.** Extend toward functional adaptor signatures, which would move the project from systems implementation toward a novel construction and a publishable contribution.

Appendix A

Code listings and reproduction

Per the writing guide, code snippets belong in an appendix rather than the body. Only the most representative listings are included here; the full source is in the project repository.

A.1 Building and reproducing the benchmarks

The evaluation is reproducible from the commands below. The upstream Dilithium reference is pinned at the repository's initial commit (`2374d22`); the classical baseline library is the BlockstreamResearch `secp256k1-zkp` `ecdsa_adaptor` module at commit `95b9835`; the Rust port builds on a vendored copy of the `fips204` crate (commit `c948882`). The C toolchain is the system compiler (`cc`, Ubuntu GCC 13.3.0) with `-O3`; the Rust toolchain is stable `rustc` (the committed run used 1.96.0) in release profile.

Listing A.1: Fair benchmark and tests (first-stage evaluation). The suite script runs everything below, saves the logs and parsed tables as a timestamped evidence run, and regenerates every figure, table, and inline number of this report from that run.

```
# the one supported evidence pipeline (all tests + all benchmarks +
    report sync)
scripts/run_benchmark_suite.sh

# or individually:
cd ref
```

```
# fair, parameter-matched base-vs-adaptor benchmark at each parameter
    set
make test/bench_levels_paper && ./test/bench_levels_paper
make test/bench_levels2 && ./test/bench_levels2
make test/bench_levels3 && ./test/bench_levels3 # target setting
make test/bench_levels5 && ./test/bench_levels5
# correctness, serialization/tamper, known-answer tests
make test/test_las3 && ./test/test_las3
make test/test_serde3 && ./test/test_serde3
make test/test_kat3 && ./test/test_kat3
# end-to-end atomic swap incl. the proof of knowledge pi
# (needs the one-time LaZer build below)
make test/test_zkp3 && ./test/test_zkp3
make test/test_swap3 && ./test/test_swap3
```

Listing A.2: Classical adaptor baseline (one-time clone).

```
git clone --depth 1 https://github.com/BlockstreamResearch/secp256k1-
    zkp \
    third_party/secp256k1-zkp # tested at commit 95b9835
cd ref && make test/bench_classical && ./test/bench_classical
```

Listing A.3: Proof-of-knowledge library (one-time clone and build; see the repository README for the dependency recipe).

```
git clone https://github.com/lazer-crypto/lazer third_party/lazer
cd third_party/lazer && make lazer.h liblazer.a
# the generated proof parameters (ref/relation_zk_params.h) are
    committed,
# so SageMath is needed only to REgenerate them, never to build
```

The Rust port is reproduced with the standard Cargo tooling; the test gate runs first because the benchmarks refuse to time unverified code, and the known-answer test asserts the same pinned digest as the C implementation (`bb6ad0da...260c`).

Listing A.4: Rust port: test gate, statistical benchmark, protocol driver, and size report (one suite script, or the four steps individually).

```
# the one supported Rust evidence pipeline (tests + benchmarks + report
    sync)
scripts/run_rust_bench_suite.sh
```

```
# or individually:
cd rust/fips204-las
cargo test --lib --tests # gate: includes the pinned KAT digest
cargo bench --bench las_bench # Criterion.rs statistical harness
cargo run --release --example bench_levels # protocol driver (mirrors
    the C driver)
cargo run --release --example size_report # packed component sizes
```

A.2 The timing-benchmark procedure

The pseudocode below is the procedure both protocol drivers implement (`ref/test/bench_level` and the Rust `examples/bench_levels.rs`, deliberately line-for-line mirrors), backing the methodology of section 2.6. Ordering matters: correctness is asserted *before* timing on the very objects about to be timed, so no failure path is ever measured, and the rejection gate runs over the *timed* calls themselves, so the gate certifies the same data the tables report. The same procedure runs once per parameter set.

Listing A.5: The timing-benchmark procedure (both tiers). $R = 5$ repetitions; $N = 500$ (sign-class) or 1000 (verify-class) iterations; the rejection gate follows eq. (2.2).

```
Input: parameter set (n, l, kappa); fixed pp seed; fixed 33-byte
   message
 1 Setup: expand A from the fixed seed; KeyGen; relation Gen (Y, y)
 2 Gate 0 (correctness): run the full adaptor contract on the exact
   objects about to be timed (PreVerify accepts; basic Verify rejects
   the pre-signature; Adapt verifies; Extract returns y exactly);
   abort the run on any failure
 3 for each operation op in {KeyGen, Sign, Verify, PreSign, PreVerify,
   Adapt, Extract}: # core tier
 4 for r in 1..R:
 5 a0 <- attempt counter # sign-class only
 6 t0 <- monotonic clock
 7 repeat N times: run op (structures in / structures out;
            fresh masking randomness inside every sign-class call)
 8 run_r <- (clock - t0) / N
```

```
 9 att_r <- (attempt counter - a0) # sign-class only
10 report mean +/- sample SD over run_1..run_R
11 Gate 1 (run validity): for Sign and for PreSign, the mean attempts
    per call over ALL R*N timed calls must satisfy
        |mean - E| <= 5 * E*sqrt(1-1/E) / sqrt(R*N)
    where E = 1/p_acc is the closed-form expectation; abort on failure
12 repeat steps 3-11 at the packed boundary (packed bytes in /
    packed bytes out, validating decode + encode inside the call)
13 emit packed sizes (communication is fixed: measured once, no SD)
```

The Criterion.rs harness measures the same operations under its own protocol (3 s warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals) and applies the same Gate 1; it shares no timing code with the drivers, which is what makes its agreement an independent check rather than a re-run.

A.3 Full benchmark data tables

The tables in chapter 3 carry the primary results; this section gives the supporting data at every parameter set, produced by the commands above (listing A.1). Table A.1 lists every core-tier per-operation timing with its standard deviation (the data plotted in fig. 3.1, and the source of the core block of table 3.2); table A.2 gives the component byte sizes at every parameter set (the target setting is the body table table 3.5).

Figure A.1 is the parameter-sensitivity check referred to in section 3.2: it repeats the primary base-versus-adaptor comparison at all four parameter sets and confirms that the adaptor premium stays in the single digits as the dimensions are scaled up, so the headline result is not specific to the target setting.

A.4 Auditing the reuse claim

The “reused vs. modified vs. added” claim (table 2.1) is verified by diffing the pristine baseline branch against the project branch.

Listing A.6: Branch diff proving the lattice core is untouched (no output means identical).

```
git diff --name-status dilithium-baseline main -- ref/
```

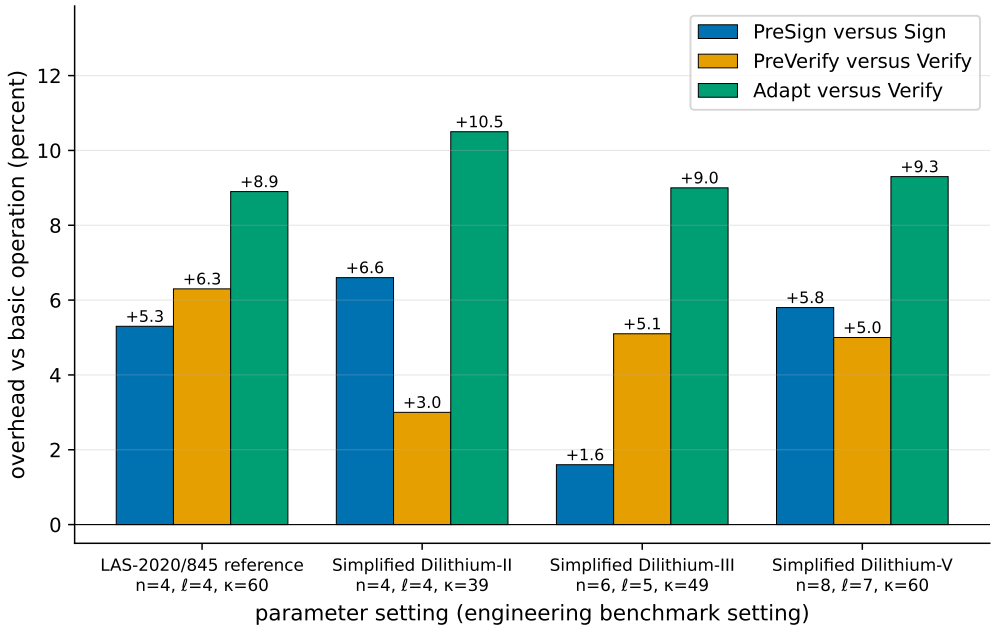

Figure A.1: Parameter sensitivity of the adaptor overhead: each LAS adaptor operation as a percentage over the basic operation it mirrors, at the four parameter sets of table 2.2. PreSign is paired with Sign; PreVerify and Adapt with Verify; Extract has no basic analogue and is omitted. The premium stays in the single digits everywhere, so adding adaptor functionality does not become more expensive as the scheme is scaled up. This is a secondary robustness check; the primary comparison is the per-operation overhead at the target setting, table 3.2.

```
git diff dilithium-baseline main -- ref/poly.c ref/ntt.c ref/sign.c \
    ref/packing.c ref/polyvec.c ref/reduce.c ref/rounding.c ref/params.h
```

A.5 Selected code listing: Adapt and Extract

The adaptor mechanism reduces to two short routines. Adapt adds the witness to the pre-signature's response; Extract recovers the witness by subtraction and confirms it against the statement. Both reuse the reference's polynomial arithmetic verbatim.

Listing A.7: `las_adapt` and `las_ext` from `ref/las.c`.

```
int las_adapt(las_sig *sig, const las_sig *presig, const uint8_t *m,
    size_t mlen,
              const las_pk *Y, const las_sk *y, const las_pk *pk, const
                  las_pp *pp) {
```

Table A.1: Per-operation timing, reported independently (μs per operation; mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500 sign-class / 1000 verify-class iterations; `bench_levels`, machine of record in section 2.6). KeyGen, Sign, and Verify *are* the basic signature and are reused unchanged by LAS; PreSign, PreVerify, Adapt, and Extract are the four adaptor operations. Public-parameter Setup, a one-time global cost, runs in 46–169 μs and is omitted. Parameter sets and symbols: tables 2.2 and 2.3.

Operation	LAS-2020/845 reference $(4, 4, 60)$	Simplified Dilithium-II $(4, 4, 39)$	Simplified Dilithium-III $(6, 5, 49)$	Simplified Dilithium-V $(8, 7, 60)$
Basic signature (reused unchanged by LAS)				
KeyGen	59.0 ± 1.7	61.7 ± 0.8	91.6 ± 2.0	133 ± 7
Sign	471 ± 24	570 ± 20	852 ± 25	991 ± 43
Verify	126 ± 3	131 ± 3	199 ± 6	265 ± 12
LAS adaptor operations				
PreSign	496 ± 21	608 ± 35	866 ± 41	1049 ± 51
PreVerify	134 ± 6	135 ± 2	210 ± 9	278 ± 10
Adapt	137 ± 7	144 ± 7	217 ± 5	290 ± 8
Extract	47.1 ± 0.7	55.6 ± 4.2	84.2 ± 6.3	117 ± 6

```
  unsigned int j;
  if(las_preverify(presig, m, mlen, Y, pk, pp))
    return -1;
  sig->c = presig->c;
  for(j = 0; j < LAS_M; ++j) { /* z = z_hat + y_witness */
    poly_add(&sig->z[j], &presig->z[j], &y->s[j]);
    poly_reduce(&sig->z[j]);
  }
  return 0;
}

int las_ext(las_sk *y, const las_sig *sig, const las_sig *presig,
            const las_pk *Y, const las_pp *pp) {
  poly Ay[LAS_N];
  unsigned int j;
  for(j = 0; j < LAS_M; ++j) { /* s = z - z_hat */
    poly_sub(&y->s[j], &sig->z[j], &presig->z[j]);
    poly_reduce(&y->s[j]);
  }
```

Table A.2: LAS objects by component (packed bytes; `bench_levels`). The LAS-2020/845 reference set shares the Simplified Dilithium-II dimensions. The challenge c is fixed; the response z grows with $n + \ell$ and dominates the signature at every setting. The target-setting column is the body table table 3.5.

Component	Simplified Dilithium-II $(4, 4)$	Simplified Dilithium-III $(6, 5)$	Simplified Dilithium-V $(8, 7)$
c (challenge)	32	32	32
z (response)	4608	6688	9120
Signature (c, z)	4640	6720	9152
z as % of signature	99.3%	99.5%	99.7%
Public key pk	2944	4416	5888
Secret key sk	512	704	960
Statement Y (LAS only)	2944	4416	5888
Witness y (LAS only)	512	704	960
Pre-signature (LAS only)	4640	6720	9152

```
  las_Amul(Ay, pp, y->s); /* check A*s == Y */
  for(j = 0; j < LAS_N; ++j)
    if(!poly_equal(&Ay[j], &Y->t[j]))
      return -1;
  return 0;
}
```

Bibliography

[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 635–664. Springer, 2021.

[2] Blockstream Research. secp256k1-zkp: Ecdsa adaptor signature module. Software library, 2024. `https://github.com/BlockstreamResearch/secp256k1-zkp`, commit 95b9835.

[3] Maxime Buser, Rafael Dowsley, Muhammed Esgin, Clémentine Gritti, Shabnam Kasra Kermanshahi, Veronika Kuchta, Jason Legrow, Joseph Liu, Raphaël Phan, Amin Sakzad, et al. A survey on exotic signatures for post-quantum blockchain: Challenges and research directions. *ACM Computing Surveys*, 55(12):1–32, 2023.

[4] CRYSTALS team. CRYSTALS-Dilithium reference implementation. Software repository, 2023. `https://github.com/pq-crystals/dilithium`, commit 2374d22.

[5] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR transactions on cryptographic hardware and embedded systems*, pages 238–268, 2018.

[6] Muhammed F Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In *European Symposium on Research in Computer Security*, pages 378–397. Springer, 2020.

[7] Lloyd Fournier. One-time verifiably encrypted signatures a.k.a. adaptor signatures. Manuscript, 2019. `https://github.com/LLFourn/one-time-VES`.

[8] Brook Heisler, Jorge Aparicio, and contributors. Criterion.rs: statistics-driven micro-benchmarking for Rust. Software repository, 2025. `https://github.com/criterion-rs/criterion.rs`, version 0.8.2.

[9] Ruslan Kysil, István András Seres, Péter Kutas, and Nándor Kelecsényi. poqeth: Efficient, post-quantum signature verification on ethereum. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 327–343, 2025.

[10] Lazer-crypto project. LaZer: a library for lattice-based zero-knowledge proofs. Software library, 2025. `https://github.com/lazer-crypto/lazer`; accompanies the CCS 2024 paper "The LaZer Library: Lattice-Based Zero Knowledge and Succinct Proofs for Quantum-Safe Privacy".

[11] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[12] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019*, 2019.

[13] National Institute of Standards, Technology (NIST), Thinh Dang, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, and Angela Robinson. Module-lattice-based digital signature standard, August 2024.

[14] Andrew Poelstra. Scriptless scripts. Presentation, Milan Bitcoin meetup / Blockstream, 2017. `https://github.com/apoelstra/scriptless-scripts`.

[15] Sebastien Riou, Jong-Yeon Park, Liga Anwar, Axel Poschmann, and Michael Hutter. Apples, oranges, and signatures: Pitfalls and methodology in ML-DSA benchmarking. Cryptology ePrint Archive, Paper 2026/1333, 2026. `https://eprint.iacr.org/2026/1333`.

[16] Eric Schorn. fips204: FIPS 204 ML-DSA in pure Rust. Software repository, 2025. `https://github.com/integritychain/fips204`, vendored at commit c948882.

[17] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.